

Вкладка 1

Отчет о лабораторной работе №7

Выполнил:
Петров Евгений Станиславович
Группа: 6204-010302D

Вкладка 2

Цель: Внести изменения в существующий набор типов табулированных функций, позволяющие обрабатывать точки функций по порядку (паттерн «Итератор»), а также выбирать тип объекта табулированной функции при его неявном создании (паттерн «Фабричный метод» и средства рефлексии).

Задачи работы:

1. Добавить поддержку цикла **for-each** для всех реализаций **TabulatedFunction**.
 2. Внедрить интерфейс фабрики и переключаемую фабрику в класс **TabulatedFunctions**.
 3. Реализовать методы создания объектов через механизм **reflection** с корректной обработкой ошибок.
 4. Подготовить демонстрационный пример, подтверждающий корректность работы всех механизмов.
-

2. Теоретическая часть

Итератор — поведенческий паттерн проектирования, позволяющий последовательно обходить элементы агрегированного объекта без раскрытия его внутренней структуры. Корректная реализация итератора требует:

- поддержания валидного внутреннего состояния;
- выбрасывания исключения **NoSuchElementException** при попытке перехода за пределы коллекции;
- генерации **UnsupportedOperationException** при вызове метода **remove**, если операция удаления не поддерживается.

Фабричный метод — порождающий паттерн, переносящий логику создания объектов в специализированные классы (фабрики). Он позволяет изменять тип создаваемых объектов без модификации клиентского кода и упрощает расширение системы.

Рефлексия в Java — механизм работы с метаданными классов (**Class**, **Constructor**, **Method**). Она позволяет:

- находить конструкторы по сигнатуре;
 - создавать экземпляры классов через **Constructor.newInstance**;
 - транслировать проверяемые исключения в более удобные для клиента, например **IllegalArgumentException**.
-

3. Методика и реализация

Среда разработки: Java.

Структура проекта:

- **functions** — базовые интерфейсы и реализации (**TabulatedFunction**, **ArrayTabulatedFunction**, **LinkedListTabulatedFunction**, **TabulatedFunctions**, фабрики);
- **functions.basic** — элементарные функции (**Sin**, **Cos** и др.);
- **functions.meta** — композиции функций;
- **threads** — классы из предыдущей лабораторной работы;
- **Main** — демонстрационный класс.

Итераторы.

Интерфейс **TabulatedFunction** расширен **Iterable<FunctionPoint>**.

В классах **ArrayTabulatedFunction** и **LinkedListTabulatedFunction** реализованы анонимные итераторы, которые:

- последовательно обходят внутренние структуры без создания промежуточных копий;
- выбрасывают **NoSuchElementException** при выходе за пределы;
- всегда выбрасывают **UnsupportedOperationException** при вызове **remove**;
- возвращают клоны **FunctionPoint**, предотвращая

нарушение инкапсуляции.

Фабрика.

Введён интерфейс `TabulatedFunctionFactory` с тремя перегруженными методами `createTabulatedFunction`. Для каждой реализации добавлены вложенные публичные фабрики:

- `ArrayTabulatedFunctionFactory`;
- `LinkedListTabulatedFunctionFactory`.

В классе `TabulatedFunctions` хранится статическая ссылка на текущую фабрику (по умолчанию — массивная). Метод `setTabulatedFunctionFactory` позволяет заменить фабрику, а все методы создания табулированных функций делегируют выполнение текущей фабрике.

Рефлексия.

В `TabulatedFunctions` реализованы перегруженные методы:

- `createTabulatedFunction(Class<? extends TabulatedFunction> clazz, ...)`;
- `tabulate(..., Class<? extends TabulatedFunction> clazz, ...)`.

Вспомогательный метод `invokeConstructor` находит конструктор требуемой сигнатуры и вызывает его. Любые ошибки (отсутствие конструктора, нарушение доступа, исключения конструктора) оборачиваются в `IllegalArgumentException`.

Демонстрация.

Класс `Main` содержит три сценария:

- `iteratorDemo` — обход точек массивной и списковой функций в цикле `for-each`;
 - `factoryDemo` — переключение фабрики с массивной на списковую;
 - `reflectionDemo` — создание функций через `Class` и табулирование функции `Sin`.
-

4. Результаты выполнения программы

Listening on 58383

User program running

=== Iterator check ===

ArrayTabulatedFunction iteration:

(0.0; 0.0)

(1.0; 1.0)

(2.0; 4.0)

LinkedListTabulatedFunction iteration:

(-1.0; 1.0)

```
(0.0; 0.0)
```

```
(1.0; 1.0)
```

```
(2.0; 4.0)
```

```
=== Factory method check ===
```

```
Default factory produced: ArrayTabulatedFunction
```

```
Switched factory produced: LinkedListTabulatedFunction
```

```
=== Reflection creation check ===
```



```
class functions.ArrayTabulatedFunction
```

```
{(0.0; 0.0), (5.0; 0.0), (10.0; 0.0)}
```

```
class functions.ArrayTabulatedFunction
```

```
{(0.0; 0.0), (10.0; 10.0)}
```

```
class functions.LinkedListTabulatedFunction
```

```
{(0.0; 0.0), (10.0; 10.0)}
```

```
class functions.LinkedListTabulatedFunction
```

```
{(0.0; 0.0), (0.3141592653589793; 0.3090169943749474),  
(0.6283185307179586; 0.5877852522924731), (0.9424777960769379;  
0.8090169943749475), (1.2566370614359172; 0.9510565162951535),  
(1.5707963267948966; 1.0), (1.8849555921538759; 0.9510565162951536),  
(2.199114857512855; 0.8090169943749475), (2.5132741228718345;  
0.5877852522924732), (2.827433388230814; 0.3090169943749475),  
(3.141592653589793; 1.2246467991473532E-16)}
```

```
User program finished
```

5. Выводы

В ходе выполнения лабораторной работы реализованы все поставленные требования:

- **табулированные функции стали итерируемыми;**
- **внедрена заменяемая фабрика для создания различных реализаций;**
- **реализовано рефлексивное создание экземпляров классов;**
- **подготовлены демонстрационные сценарии, подтверждающие корректность работы паттернов.**